

Sample Chapter: Core JDO ISBN 0-13-140731-7

The chapters of the book titled “Core JDO” are being made available for the purposes of review to allow the community to participate in the review and editing of the chapters of the book.

Copyright (c) 2002 Sun Microsystems, Inc. All rights reserved.

FOR PRIVATE USE ONLY: This material is the property of Sun Microsystems, Inc. and is not to be sold, reproduced in any form by any means, or generally distributed without the prior written authorization of Sun Microsystems Inc and Prentice Hall.

To add your comments

- Simply send your uninhibited comments and feedback to the authors at jdoobook@yahoogroups.com
- Or click and add a note using Acrobat where needed. E-mail the PDF back to the authors at jdoobook@yahoogroups.com
- If you would like to receive the chapters in an alternate format, have any questions, concerns or please contact the authors at jdoobook@yahoogroups.com

"A mistake is to commit a misunderstanding"- Bob Dylan

Transactions

INTRODUCTION

The ability for tasks to be completed predictably is an integral part of any real world application. The transaction facilities in the middleware or server side applications are responsible for ensuring this predictability. In this chapter we will begin by examining some basic concepts surrounding transactions and how transactions are used in Java. We will then look at how the concept of transactions in JDO and the more complicated scenarios resulting from integration into managed environments ¹⁾

Comment: Footnote to definition or reference to chapter 1. JDO overview

TRANSACTION CONCEPTS

A transaction can be described as a *unit of work* that is comprised of several different operations acting on one or more shared system resources and has a predictable outcome. Additionally this unit of work needs to be governed by *ACID* properties. The acronym (ACID) stands for Atomic Consistent, Isolated and Durable. These four classical characteristics essentially define the concept of a transaction:

- Atomic: The unit of work is atomic, in that it is either performed in its entirety or not performed at all. This all or nothing properly ensures that in case of failure changes that occur due to the unit of work are either made permanent (committed) or reversed (rolled back) to restore the original state.
- Consistent: Consistency requires that after the execution of the unit of work the system is in a stable state. If there is any inconsistency then the work done must be reversed to

Comment: Andreas Reuter's classic Transaction Processing: Concepts and Techniques (ISBN 1 -55860-190-2) Morgan Kauffman

restore the original state. For example the referential integrity of the data must be maintained upon completion.

- **Isolated:** The notion of isolation is simple, a transaction should execute as though it was running by itself, meaning that the correctness of the unit of work performed in a transaction should not be compromised when another concurrent transaction accesses the same set of resources.
- **Durable:** The changes made by the unit of work constituting the transaction should be permanent and should not be lost because of system failure.

From a design perspective a transaction will usually represent a higher level of abstraction. A transaction will consist of business operations that have application semantics and a scope greater than a method invocation or database operation. For example the transaction will consist of an account withdrawal and an account deposit, the application code to realize this may consist of multiple databases and classes. However the overall transaction is still governed by the ACID properties which require that all the work done in the transaction be completed as one entity or in the case of failure no work is done at all, with data integrity being maintained in both cases. This all-or-nothing *flat* transaction model is characterized with the states shown in Figure 1. The transaction is begun and goes into an active state where some work is done. If the work was successfully performed the transaction is committed and the resources accessed in that transaction updated with the results of that work. If something goes wrong the transaction is rolled back and the resources restored to their initial state.

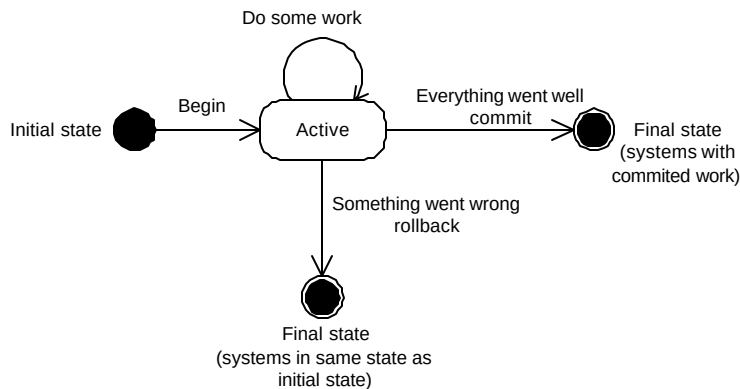


Figure 1: State transitions in a transaction

The nested transaction model allowing transactions to be nested inside each other is not supported by J2EE or JDO.

Parties involved

In any transactional system there are multiple parties that work in unison for the model work smoothly. These are

- Transactional object: This is an object that initiates or demarcates a transaction. It can be any user application or in an application server environment a J2EE component like an EJB
 - Transaction manager: This is the implementation of a transaction service and is responsible for coordinating and managing the transactions.
 - Resource adaptor: This is the system level software “driver” that application code uses to connect to the underlying resources. E.g. JDBC drivers, JMS providers
- Resource manager: A resource manager manages a set of resources. A transactional resource manager can participate in transactions that are externally controlled and coordinated by a transaction manager. A resource manager is typically in a different address space or on a different machine from the resource adaptors and clients that accesses it. E.g. Database system, a mainframe transaction processing system etc.
- Resource: This is the actual underlying resource. E.g. a database instance

These are also shown in Figure 3 later. Depending on the shared resources being accessed by the transactional object the transaction can be characterized as local or a distributed transaction.

Local transactions

A transaction that involves a single resource manager is known as a local transaction. In local transactions, the transaction manager simply delegates management of the transaction to the underlying resource manager. For example in an application server when a transaction object like a J2EE component interacts with a single database, the transaction manager creates a transaction object, associates it with the current thread and invokes a `begin()` `commit()` and `rollback()` on the underlying database connection.

Distributed transactions

A distributed transaction is a transaction that interactions with multiple and possibly heterogeneous resource managers in a coordinated manner. For example a transaction manager will need to coordinate the transaction involving a database and a JMS queue in the same method execution to ensure that these resources work together coherently.

From a JDO perspective, the concept of distributed transactions is important if the application attempts to use JDO based persistence alongside application code that uses other persistence mechanisms.

Two phase commit

In order to maintain the atomicity of a distributed transaction the transaction managers use a standard two level recovery mechanism known as a two-phase commit protocol (2PC). The two-phase commit protocol guarantees data integrity by ensuring that transactional updates are committed in all of the participating resources, or are fully rolled back out of all the resources, reverting to the state prior to the start of the transaction. To understand how 2PC works, let us look at what happens in such a transaction.

1. The resources are enlisted with the transaction manager.
2. The transactional object initiates a transaction
3. The transaction manager begins the transaction.
4. Application code updates resource A
5. Application code updates resource B
6. Application code requests a commit from the transaction manager
7. The transaction manager executes the two phase commit.

The transaction manager, as the name suggests, completes the transaction in two phases. A simplified view of the interaction is shown in Figure 2.

Distributed transaction standards:

The most widely accepted distributed transaction processing model is that specified by the Open-group consortium in the XA specification. The XA specification defines the API in the form of an interface – the `XAI`nterface that serves as a contract between distributed transactional components. See <http://www.opengroup.org>

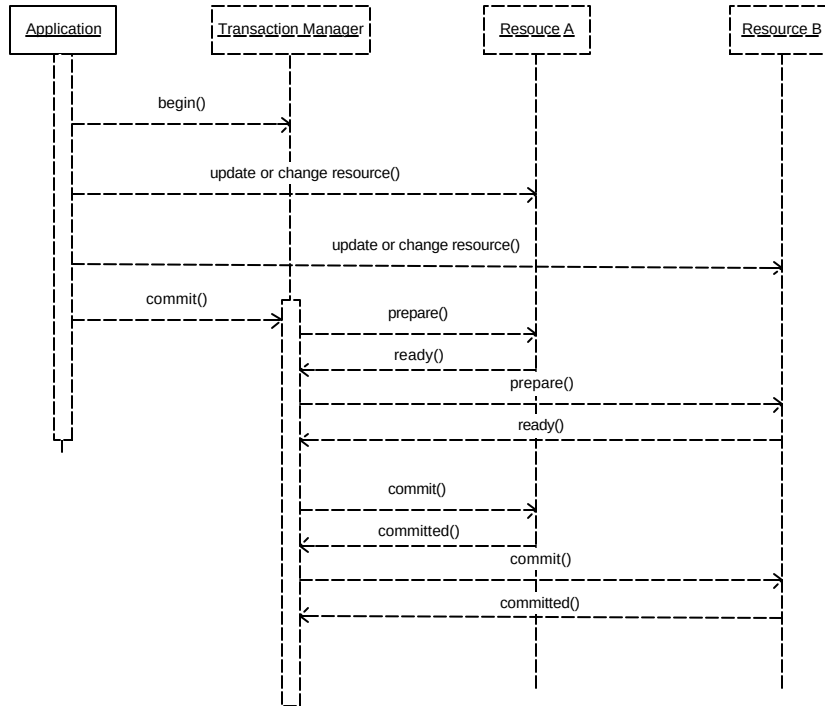


Figure 2 : Two phase commit execution

1. Phase 1: The transaction manager send a *prepare-to-commit* request to the resource managers. The resource managers perform internal checks and store information about the requested updates in some durable storage (e.g. like disk). When the transaction manager receives acknowledgements from all the resource managers, it itself stores the information somewhere like in a file system.
2. Phase 2: The transaction manager sends a commit message to all the resource managers. The resource managers perform the requested update on their end and send an acknowledgement of the execution back to the transaction manager. Once the transaction manager receives all the acknowledgements it informs the transactional object about the result of the transaction.

The transaction manager will issue a rollback message to the resource managers if:

- Any of the resource managers are unable to prepare themselves (e.g. locks could not be obtained) and return a failure message in phase one.
- Any of the resource managers are unable to complete the task and return a failure message in phase two.

The two phase commit protocol in distributed transactions can sometimes give rise to fatal error conditions called *heuristics* that violate the *atomic* properties. A typical case called a *heuristic rollback* is as follows:

In phase one, the transaction manager signals the resource to prepare itself within the context of a transaction, the acknowledgement from the resources is in fact a commitment that later on when the transaction manager signals a commit or rollback, it will be able to do either. However sometimes it may break this promise— for example the transaction manager goes away and the resource is waiting in a prepared state tying up other locks. Even though the resource does not know the outcome of the transaction it will make a preemptive decision, called a *heuristic decision*—based on some internal timer or other administrative command. Typically this guess is to rollback the transaction, release internal locks and return itself to a consistent state. If subsequently the transaction manager returns with a request to do some work (e.g. commit) on the same transaction the resource will throw a heuristic exception. Some other similar heuristic conditions are:

- **Heuristic commit:** A resource preemptively decides to commit its part of a distributed transaction.
- **Heuristic mixed:** Due to one or more heuristic decisions a transaction has different branches in conflicting states, some committed and some rolled back.

From a developers perspective, any indication of a heuristic condition it is a signal to examine the application deployment and architecture. Typical causes may include a lack of system resources on the database or resource manager and excessive consumption of resources (e.g. locks or memory) by a particular transaction.

TRANSACTIONS IN JAVA

JDBC Transactions

Java applications can include transactional constructs at different levels. An application interacting with a datastore using JDBC can use JDBC level transactions. A JDBC transaction is controlled by the transaction manager of the underlying datastore and is demarcated using the commit and rollback methods of the `java.sql.Connection` interface. For example:

```
Connection conn = DriverManager.getConnection
    ("jdbc:oracle:thin:@jdo:Oracle");
```

```

conn.setAutoCommit(false);
try{
    Statement stmt = con.createStatement();
    stmt.executeUpdate("UPDATE USERS SET username='corejdo'
                        WHERE ACCOUNTID=1829");
    stmt.executeUpdate("UPDATE ACCOUNTS set ACCOUNTTYPE=09
                        WHERE ACCOUNTID=1829");
    conn.commit();
}catch(Exception e){
    conn.rollback();
}

```

JDBC transactions while simple are not suitable for controlling transactions that span multiple application components. The connection based demarcation is local to the component in which it is used and the transaction context is not propagated. For distributed environments like J2EE a different framework is needed that will cater to transactional resources in general, not just databases. This is where JTA and JTS come in.

JTS and JTA

The Java Transaction API (JTA) and Java Transaction Service (JTS) provide the necessary and generic framework for applications to use transactions across multiple components and data stores.

The OMG Object Transaction Service (OTS) that specifies how a transaction service can be implemented and the associated IDL interfaces. OTS is the transaction service used in CORBA to provide the transaction related plumbing. JTS is the Java mapping of the IDL interfaces that defines how the service should be implemented in Java and the functionality that it must expose. Vendors (like J2EE application server vendors) use and provide the implementation of the JTS in their products, developer never work directly with JTS.

Java Transaction API (JTA) provides the API and the standard Java interface for developers to access the facilities of a transaction service (i.e. the JTS implementation). It provides an interface for the transaction manager, which manages the underlying transaction service. Figure 3 summarizes how JTA and JTS work together.

- Transactional objects like EJBs and Java classes obtain a reference to the instance of the JTA `UserTransaction` interface and demarcate transactions.
- Vendors provide the JTS implementation and a transaction manager implementation. This is exposed using the `JTATransactionManager` interface.
- When a transaction is begun by the transactional object, the transaction manager creates an instance of the transaction object, represented by the `JTA Transaction` interface and associates it with the calling thread.

- When the transactional object uses the transactional resource the resource is enlisted with the transaction manager

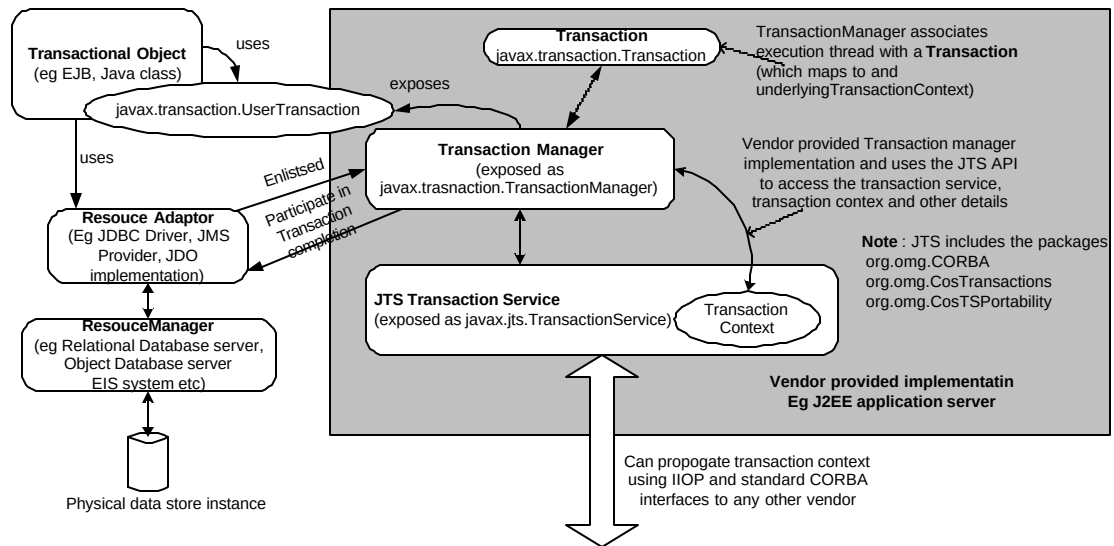


Figure 3 : Different entities in JTA

Enlistment refers to the process of making a resource a participant in a specific transaction and occurs in two flavors

Static Enlistment: The application server automatically enlists the resource in any transactions it handles before any application code executes e.g. at start up.

Dynamic Enlistment: Some container implementations are smart enough realize that enlistment is not needed until the resource is used and enlist the resource dynamically when it is accessed.

The code below shows how the `UserTransaction` can be used in an application

```

UserTransaction tx = // lookup user transaction object
tx.begin();
// do some work across multiple components and datastores
tx.rollback();

```

In the above code the reference to the `UserTransaction` can be obtained from JNDI context of the the J2EE application or using the EJB context if the component is an EJB. For

example:

```
Properties p= new Properties();
p.put(Context.INITIAL_CONTEXT_FACTORY,"some.vendor.factor.c
lass");
p.put(Context.PROVIDER_URL, "applciation serve URL");
InitialContext ic = new InitialContext(p);
UserTransaction tx =
UserTransaction)ic.lookup("javax.transaction.UserTransactio
n");
//Or in EJB's
UserTransaction tx = ejbContext.getUserTransaction() ;
```

The `UserTransaction` is simple and is only used to mark the transactional boundary. The application code never has access to the actual `Transaction` object. In this sense the `UserTransaction` name is a misnomer since it works as more of a user transaction manager. It does not represent a single transaction, but the applications ability to begin and commit transactions. Table 1 lists some of the important methods in the `UserTrasnaction` interface.

UserTransaction method	Description
<code>void begin()</code>	Tells the transaction manager to create a new transaction and associate it with the current thread.
<code>void commit()</code> <code>int getStatus()</code>	Commit and make permanent the changes made in the transaction. Get the status of the transaction associated with the current thread
<code>void rollback()</code>	Rollback and undo the changes made in the transaction.
<code>void setRollbackOnly()</code>	When this in invoked the transaction manager is informed that the only possible outcome of the transaction is to undo and roll back the work done in the transaction.
<code>Void setTransactionTimeout(int seconds)</code>	Set the timeout value

Table 1: Methods in the `UserTransaction`

The interface `javax.transaction.TransactionManager` has methods identical to the `UserTransaction` with the additional ability to return the `Transaction` object. The `TransactionManager` also has two additional methods that allow the transaction to be suspended and resumed. These are listed in Table 2. J2EE does not mandate that a container expose the `TransactionManager` interface to application code (application code always works with `UserTransaction`), therefore some containers may not expose this. Typically containers will bind the `TransactionManager` to a specific JNDI name for their product.

TransactionManager methods	Description
<code>Transaction getTransaction()</code>	Returns the actual <code>Transaction</code> object associated with the current thread.
<code>Transaction suspend()</code>	Suspend the current transaction
<code>void resume(Transaction tobj)</code>	Resume execution of the current transaction.

Table 2: Additional methods in the `TransactionManager`

The `javax.transaction.Transaction` object is used by the `TransactionManager` as an abstraction to the actual executing transaction. Application code almost never accesses the actual `Transaction` object. It is here that the resource enlistment and de-enlistment takes place. Table 3 summarizes the methods in this interface.

Transaction methods	Description
<code>boolean enlistResource(XAResource xares)</code>	Invoked by the <code>TransactionManager</code> to enlist the resource with the transaction
<code>boolean delistResource(XAResource xares, int flag)</code>	Invoked by the <code>TransactionManager</code> to delist the resource with the transaction
<code>void registerSynchronization(Synchronization sync)</code>	An object implementing the <code>javax.transaction.Synchronization</code> interface can be registered with the <code>Transaction</code> to receive callbacks. The <code>TransactionManager</code> will invoke the <code>beforeCompletion()</code> method in the passed object before starting the transaction commit

	process and the <code>afterCompletion()</code> method when the transaction has completed.
--	---

Table 3: The methods in the `javax.transaction.Transaction` interface

Now that we have an overall understanding of how transactions work and how they are abstracted by JTA and JTS for use in Java and J2EE, let us look at how JDO deals with transactions.

TRANSACTIONS IN JDO

Earlier in this chapter we briefly mentioned that in a flat transaction model the calling thread is associated with a transaction. In JDO the `PersistenceManager` represents the applications view of any persistent data. Logically it is here that this association must occur. To separate out the functionality from the `PersistenceManager` itself, JDO defines a `javax.jdo.Transaction` interface. This `Transaction` has a one to one relationship with the `PersistenceManager`, meaning that if parallel transactions execution is required in an application then multiple persistence managers need to be obtained from the factory.)

Comment: In Chapter 1 we introduced the concept of transient and persistent instances.

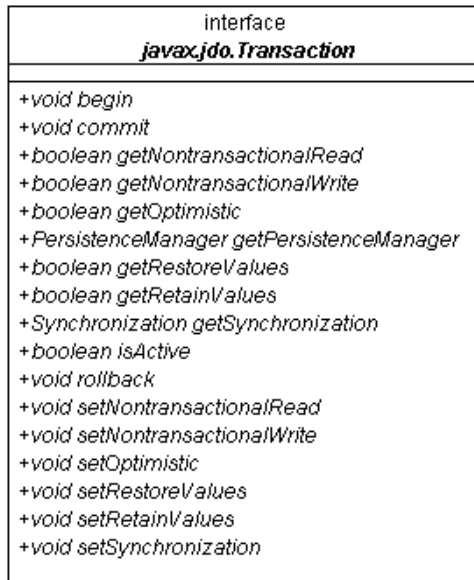


Figure 4: The `javax.jdo.Transaction` interface

The Transaction interface by itself is quite simple and the class diagram is shown in Figure 4. It contains a few different get and set methods that can be used to access and set different properties of the Transaction instance. These methods are discussed in subsequent sections. The `begin()`, `commit()` and `rollback()` are the primary methods that application code uses to demarcate the transaction.

Any persistence related work or interaction with persistent objects done by the application must be performed in the context of a JDO transaction as shown below:

```

PersistenceManager pm = pmf.getPersistenceManager();
Transaction tx = pm.currentTransaction();
tx.begin();
// Do some work here with persistent objects
tx.commit();
// OR tx.rollback()

```

To preserve isolation semantics (the “I” in the ACID) a transaction will “lock” the data in the underlying datastore using different concurrency control techniques. The locking, which happens under the covers and completely transparent to application code, ensures that multiple operations on the same data set do not interfere with each other. This is analogous to the way the `synchronized` keyword operates in the Java language. By using different

locks (e.g. read locks, write locks etc) the transaction ensures that the serializable characteristic or the appearance that the transactions are acting upon the data set one after the other is maintained.

Most resource managers allow four isolation levels for a transaction which in increasing order of strictness are:

- o Read Uncommitted: If another concurrently executing transaction writes data to the underlying resource without committing it, the data is visible in the executing transaction
- o Read Committed: Committed data from another concurrently executing transaction is available
- o Repeatable Read: Data read, which has been committed by another concurrently executing transaction is guaranteed to retain the same values, when read again.
- o Serializable: Guarantees that two transactions operating on the same data execute serially one after the other.

The only way to specify isolation levels in J2EE (including EJBs), is using resource manager specific API like the JDBC Connection API. Most relational database by default utilizes the repeatable read or serializable isolation level. Serializable isolation solves the problem of unrepeatable reads (application reads data and works on it but when re-read the data set is different because of modification by another concurrently executing transaction) and phantom reads (application reads data set, when re-read there is addition to the data set due to modifications by another concurrently executing transaction) but can degrade performance due to the large number of locks required.

Based on the locking strategy and isolation level used a transaction can be categorized into two categories:

PESSIMISTIC TRANSACTIONS

The data that is being used in this transaction is locked in the underlying resource and cannot be modified by anybody until the transaction has completed. This corresponds to the serializable isolation level and is also the strategy that the EJB model assumes. Pessimistic transactions are the default model for JDO that all vendors must implement. When a transaction executes the data reflected in the persistent objects is locked by the underlying datastore.

```
PersistenceManager pm = pmf.getPersistenceManager();
Transaction tx = pm.currentTransaction();
tx.begin();
Author author = new Author("Keiron McCammon");
pm.makePersistent(author);
// do some other work here
```

```
// the data in the datastore that represents the author
// object is locked
tx.commit()
// OR tx.rollback()
```

OPTIMISTIC TRANSACTION

This model assumes that it is unlikely for the data being used to be modified by any other transaction in the application. If at commit time, the datastore detects a collision then the transaction is rolled back. This is an optional strategy for JDO vendors to implement. The JDO transaction strategy can be specified using the `setOptimistic(boolean optimistic)` method. When invoked with a `true`, the transaction is set to be optimistic, a `false` results in the default pessimistic strategy.

In order to understand this further let us look at the traditional producer-consumer example built on the code introduced in Chapter X. The “hungry” consumer is shown in Listing 1 runs in an infinite loop and searches the datastore for a particular object and transaction type is set with an argument. The producer creates the object instance, saves it and reads the data back instantly as shown in Listing 2.

Comment: Chapter 3, Developing with JDO

```
import com.corejdo.examples.chapter4.model.Author;
import javax.jdo.*;
import java.util.*;
import java.io.FileInputStream;

public class Consumer {
    public static void main (String args[]) throws Exception{
        boolean txtype= new Boolean(args[0]).booleanValue();
        Properties p = new Properties();
        p.load(new FileInputStream("config.properties"));
        PersistenceManagerFactory pmf =
            JDOHelper.getPersistenceManagerFactory( p );
        PersistenceManager pm = pmf.getPersistenceManager();
        System.out.println("Reader searching..");
        while(true){
            Transaction tx = pm.currentTransaction();
            tx.setOptimistic(txtype);
            tx.begin();
            Query query = pm.newQuery(Author.class);
            String filter = "name ==(\"Keiron McCammon\")";
            query.setFilter(filter);
            Collection authors =(Collection) query.execute();
```

```

        Iterator it = authors.iterator();
        while (it.hasNext()) {
            Author author = (Author) it.next();
            String name = author.getName();
            System.out.println("Author name = " +name);
        }
        tx.commit() ;
    }
}

```

Listing 1: The hungry Consumer

```

import com.corejdo.examples.chapter4.model.Author;
import javax.jdo.*;
import java.util.*;
import java.io.FileInputStream;

public class Producer {
    public static void main(String[] args) throws Exception {
        boolean txttype= new Boolean(args[0]).booleanValue();
        Properties p = new Properties();
        p.load(new FileInputStream("config.properties"));
        PersistenceManagerFactory pmf =
            JDOHelper.getPersistenceManagerFactory( p );
        PersistenceManager pm = pmf.getPersistenceManager();
        // create and persist the object
        Transaction tx = pm.currentTransaction();
        tx.setOptimistic(txttype);
        tx.begin();
        Author author = new Author("Keiron McCammon");
        pm.makePersistent(author);
        System.out.println("Author persisted");
        tx.commit() ;

        // Read the persisted object from the datastore
        tx.begin() ;
        Query query = pm.newQuery(Author.class);
        String filter = "name ==(\"Keiron McCammon\")";
        query.setFilter(filter);
        Collection authors = (Collection) query.execute();
    }
}

```

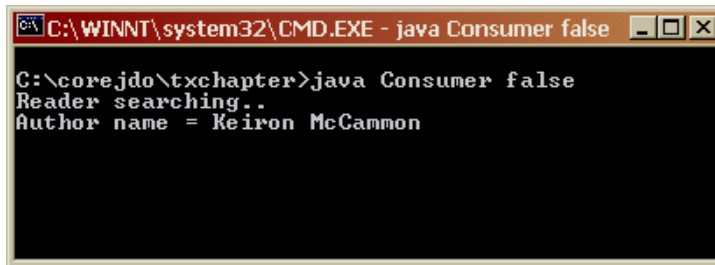


```
        System.out.println("Everything is locked");
        // do some other work here
        Thread.sleep(120000);
        // the data in the datastore that represents the author
        System.out.println("Producer wakeup");
        tx.commit() ;
        System.out.println("Producer closed");
        pm.close();
    }
}
```

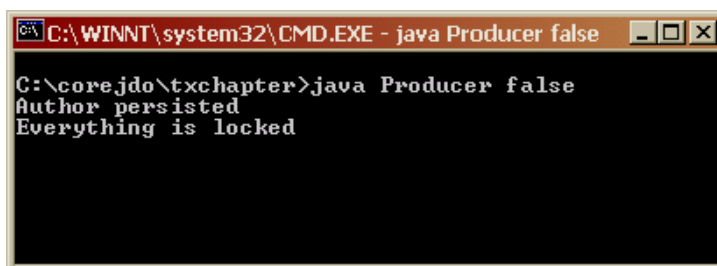
Listing 2: The Producer

Follow all the steps all the steps outlined in Chapter X are followed (i.e. the classes compiled, enhanced and the datastore set up for the implementation being used) and then first start the consumer followed by the producer.

With pessimistic or datastore transactions, the producer will lock the data corresponding to the persistent objects in the underlying datastore making it inaccessible to other transactions. The consumer will only be able to access the data in the window of time between the two transactions in the producer or when the producer exits. This is shown in Figure 5 a and b.



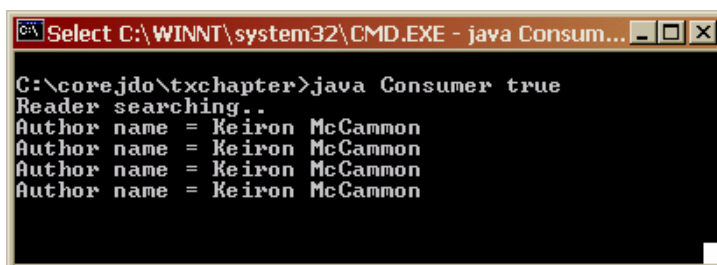
```
C:\WINNT\system32\CMD.EXE - java Consumer false
C:\corejdo\txchapter>java Consumer false
Reader searching..
Author name = Keiron McCammon
```



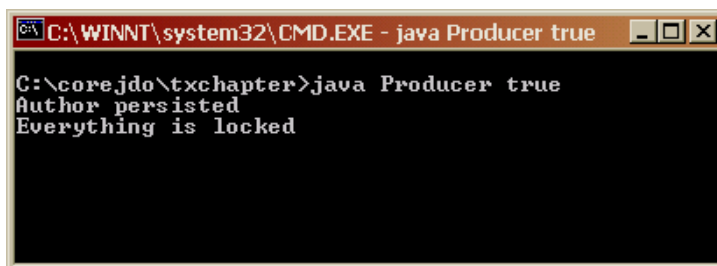
```
C:\WINNT\system32\CMD.EXE - java Producer false
C:\corejdo\txchapter>java Producer false
Author persisted
Everything is locked
```

Figure 5 a and 5 b : Producer has locked the data in pessimistic transaction and the consumer is waiting for the locks to be released.

With optimistic transactions, the producer will not lock the data for the duration of the transaction, the consumer is therefore able to access the data immediately after it is persisted as shown in Figure 6 a and b.



```
Select C:\WINNT\system32\CMD.EXE - java Consum...
C:\corejdo\txchapter>java Consumer true
Reader searching..
Author name = Keiron McCammon
Author name = Keiron McCammon
Author name = Keiron McCammon
Author name = Keiron McCammon
```



```
C:\WINNT\system32\CMD.EXE - java Producer true
C:\corejdo\txchapter>java Producer true
Author persisted
Everything is locked
```

Figure 6 a and 6 b: The optimistic producer does not lock the data and the consumer access it immediately.

There are distinct advantages and disadvantages to each locking technique. Table 4 and Table 5 summarize the differences and approach to use.

Strategy	Advantages	Disadvantages
Pessimistic : Data is locked until the transaction completes	Provides highest degree of consistency in the data.	- Does not scale well - May cause deadlocks when transactions are waiting on each other to release locks - May be detrimental to performance because of the high degree of locking
When to use	- When concurrent access is rare or not expected - When transactions are short lived - When user think time does not affect transaction duration.	
When not to use	- For long lived transactions - If optimistic transactions suffice (see Table 5)	

Comment: Any other reasons ??

Table 4: Pessimistic Transactions summarized

Strategy	Advantages	Disadvantages
Optimistic: Data is locked only when the transaction committed.	- Performs better since locking overhead is reduced - Systems generally scale better since resource usage at the datastore level is reduced	Requires developers to write code to support collision detection and handling such situations
When to use	- When concurrent access is expected to be frequent and high - When transactions are long lived - When the clients are remote (e.g. the JDO code on thick	

	clients)
	• When transaction involve user think time
When not to use	• For long lived transactions

Comment: Any other reasons?

Table 5: Optimistic transactions summarized

Understanding conflicts with optimistic transactions

Though optimistic transactions assume that it is unlikely for the underlying data to be modified it is being used, such situations can occur. To understand how to resolve such potential conflicts consider the scenario in Figure 7 where two overlapping transactions, Transaction A and Transaction B, affect the same underlying data.

When transaction B attempts to commit its changes at time t_2 it would be forced to rollback since the underlying data has been changed externally by Transaction A at t_1 (the data store will detect a collision). This is often called the “*First Commit Wins*” in which the first set of changes are not overwritten and force the second transaction to restart the work with the changed data. The exception facilitates failure detection; however the amount of rework is increased since developers now have to explicitly account for such situations and retry Transaction B and the business logic by reloading the data from the data store.

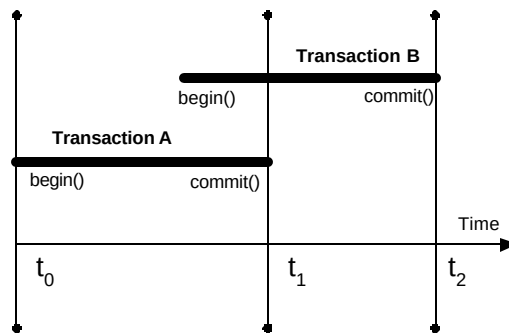


Figure 7: Conflicts in optimistic transactions

One strategy for reducing the occurrence of such conflicts is called “*Last Commit Wins*” which assumes that the consequences of overwriting data are acceptable to the application. This strategy tries to reduce the risk of overwrites by refreshing the data from the underlying data store in Transaction B just prior to performing the business logic. In JDO this is achieved through the `refresh()` method in the `PersistenceManager` which reloads

the state a particular persistent object from the datastore. For example in the code below the `author` instance is refreshed explicitly in an optimistic transaction.

```

        PersistenceManager pm = pmf.getPersistenceManager();
        Transaction tx = pm.currentTransaction();
        tx.setOptimistic(true);
        tx.begin();
        try {
            //
            // do some work here
            //
            Author author = (Author) it.next();
            String name = author.getName();
// Just before changing the persistent object refresh it
            pm.refresh(author);
            author.setName("John Malkovich");
            tx.commit();
        }catch(Exception e){
            if(tx.isActive())
                tx.rollback();
        }
    }

```

Transactional and Non transactional objects

In Chapter 3 we mentioned how the `PersistenceManager` includes a caching mechanism and in Chapter 4 we looked at the different states including transactional and non transactional states.

To recall, objects can be categorized as transactional and non transaction depending on the caching behavior of the persistence manager. This is orthogonal to the concept of transient and persistent objects introduced in Chapter 1 and later covered as states in Chapter 4. A transactional object is an object whose *managed* fields are cached by the underlying implementation of the `PersistenceManager` in the context of a `Transaction`. For example consider the code below:

```

PersistenceManager pm = pmf.getPersistenceManager();
Transaction tx = pm.currentTransaction();
tx.begin();
Author author = new Author("Keiron McCammon");
try {
    author.setName("John Malkovich");
    // do some other work here
}

```

```
tx.commit()
} catch (Exception e) {
    tx.rollback();
}
```

Object instance variables that are persisted are called *persistent fields*; instance variables that participate in a transaction are called *transactional fields*, meaning that their values can be restored by a rollback. Both persistent and transactional fields are collectively called as *managed fields* since they need to be managed by the JDO implementation.

By default the “name” field will be transactional because the `Author` class is persistence capable and the value “Keiron McCammon” will be cached in the `PersistenceManager` cache. This value will be restored in case of a potential `rollback()` and discarded when `commit()` is invoked.

All this is unrelated to the fact that the `Author` object is transient or persistent. In this example the instance is persistent and so will undergo a state change to persistent-dirty when the `setName()` method is invoked. However `author` could be an instance of a transient object as well, in which case the state changes would be different as explained in [Chapter X](#).

Recall from [Chapter X](#) that individual fields can be marked as *persistent* or *transactional* in the metadata descriptor. By default fields that are not static, final or transient are marked as “persistent” and exhibit transactional behavior in the enhanced class. When marked as “transactional” the fields will be cached and restored on a transactional rollback but will not be persisted to the data store.

In summary both transient and persistent objects can be transactional or non transactional. JDO requires that implementations support transactional behavior for persistent objects. A transient transactional object would be one that by itself does not represent persistent data, but has fields that recognize transactional boundaries and are restored to their original values by the JDO implementation when the transaction rolls back. Support for transactional behavior in transient objects is optional, meaning a JDO implementation is not required to cache fields for a transient objects.

This is in line with the general behavior where between the scope of a `tx.begin()` and a `tx.commit()` transient objects (i.e. persistence capable instances that are made transient using the `pm.makeTransient()`) are not managed by the JDO implementation. Therefore by default transient instances are non-transactional whereas persistent instances are

managed by the JDO implementation and are transactional.

A transient object is merely an instance that is not persisted however the class definition must still be persistence capable in order for JDO to manage the state transitions.

	Transac- tional	Non-Transactional
Persistent Ob- jects	Required	Optional
Transient objects	Optional	Required (JVM's default be- havior)

Table 6: Transactional behavior in persistent and transient objects

The persistence manager can be configured with two optional properties that specify how non transactional instances are accessed and can be very helpful while using optimistic transactions.

- `javax.jdo.option.NontransactionalWrite`: When this property is set to `true` the fields of a persistent non-transactional instance can be modified outside the scope of a JDO transaction even though changes made in this manner are not automatically propagated to the datastore. Without this property explicitly set the default behavior is to throw a `JDOUserException` for any attempt to modify a persistent instance outside the scope of a JDO transaction.
- `javax.jdo.option.NontransactionalRead`: When this property is set to `true` the fields of a non transactional instance can be read outside the transaction.

The usage of both these properties can be better understood by the example in Listing 3. The properties file passed to the `JDOHelper` must contain the above properties set to `true` and the implementation must support this behavior which is optional in the specifications. The example first creates and saves an object. It then uses the `NonTrasnactionalRead` property to read it outside the JDO transaction and the `NonTrasnactionalWrite` property to modify the object. The example also shows transactional behavior of transient and persistent instances.

```
import com.corejdo.examples.chapter4.model.Author;
import javax.jdo.*;
import java.util.*;
```

```
import java.io.FileInputStream;

public class NonTxReadWriteExample {

    public static void main(String[] args) throws Exception{

// The properties file contains all configuration
// properties
        Properties p = new Properties();
        p.load(new FileInputStream("config.properties"));
        PersistenceManagerFactory pmf =
JDOHelper.getPersistenceManagerFactory( p );
        PersistenceManager pm = pmf.getPersistenceManager();

// create and persist the object
        Transaction tx = pm.currentTransaction();
        tx.setOptimistic(true);
        tx.begin();
        Author author = new Author("Keiron McCammon");
        pm.makePersistent(author);
        System.out.println("Author persisted");
        tx.commit() ;
// At this point the data is committed to datastore.

// Note Below line with throw javax.jdo.JDOUserException //
// if implementation does not support property
// javax.jdo.option.NontransactionalRead=true

        System.out.println("Author Name = "+author.getName());

// Persisted data of Keiron above will not be be
// overwritten by this even though the instance is modified
// and available in memory while this application is
// executing.

// Note :Below line with throw javax.jdo.JDOUserException
// if implementation does not support property
// javax.jdo.option.NontransactionalWrite=true

        author.setName("Heiko Bobzin");

// Create another object. This is transient
        Author contributingauthor = new Author("John Malkovich");
```



```
        tx.begin();
// Transient-Transactional
        pm.makeTransactional(contributingauthor);
// make some changes to object
        contributingauthor.setName("Jeanne Smith");
// objects state will be restored from cache
        tx.rollback();

// should not print out "Jeanne Smith" but initial cached
// value of "John Malkovich"
        System.out.println("Contributing Author Name = "
+contributingauthor.getName());

// the segment below shows transactional behavior in the //
persistent objects.
        tx.begin();
        Author anotherauthor = new Author("John Doe");
// instance is transient
        pm.makePersistent(anotherauthor);
// instance is now persistent
        anotherauthor.setName("Michael Vorburger");
        System.out.println("Author name inside tx="
+anotherauthor.getName());
        tx.rollback() ;

// The cached value of John Doe is replaced in the
// persistent object.
        System.out.println("Author name outside tx="
+anotherauthor.getName());

// Read the persisted object from the datastore
        Query query = pm.newQuery(Author.class);
        Collection authors = (Collection) query.execute();
        Iterator it = authors.iterator();
        while (it.hasNext()) {
            author = (Author) it.next();
            String name = author.getName();
            System.out.println("Author name = " + name);
        }
    }
}
```

Listing 3: Non transactional read-write example

Retaining and Restoring values

The `PersistenceManager` also allows two optional settings that affect the manner in which instances are kept in memory at commit and rollback time.

- `javax.jdo.option.RetainValues`: By default once a transaction commits, the persistent instances in the cache are evicted and moved to a hollow state as explained in [Chapter X](#). This allows the implementation to easily manage the cache by keeping it small. When this property is set to `true` the instances are retained in the cache after a transaction commits. This can increase the cache size but can be helpful if the same instances are reused across transactions in an application.
- `javax.jdo.option.RestoreValues`: By setting this property to `false` the default transactional behavior of restoring fields to the state they were in when the `makePersistent()` method was invoked can be overridden. Application code will need to explicitly restore the original values. Since the overhead of the cache is removed, the performance is generally improved with this property. This should only be set if instances are not reused after a transaction rolls back. The code segment below from the previous example explains this further.

```
Transaction tx = pm.currentTransaction();
// the segment below shows transactional behavior in the
// persistent objects.
tx.begin();
    Author anotherauthor = new Author("John Doe");
    pm.makePersistent(anotherauthor);
    anotherauthor.setName("Michael Vorburger");
    System.out.println("Author name inside tx="
                      +anotherauthor.getName());

tx.rollback() ;
// With javax.jdo.option.RestoreValues=false this will not
// restore the value to "John Doe" since that was never
// cached and the value "Michael Vorburger" will be printed

System.out.println("Author name outside tx="
                  +anotherauthor.getName());
```

The optional features `RetainValues` or `RestoreValues` only affect how the instances are kept in memory at commit or rollback time respectively. They do not alter the outcome of the commit or rollback on a transaction. The `RestoreValues` option can be used if instances are not used across rollbacks and the `RetainValues` option if instances are used across transactions.

JDO and Transactions in a J2EE application server

In order to incorporate transactional semantics in J2EE components that utilize JDO for transparent persistence it is important to understand the how a transaction in JDO behaves in such managed environments.

As mentioned in Chapter 1 and [X](#) the JDO specifications define the relationship and integration between the JDO and the managed J2EE environment by requiring the vendors to satisfy the JCA contracts. In short the JDO implementation is integrated into the J2EE container as a JCA connector (i.e. a RAR file). The components use JDO by obtaining the `PersistenceManagerFactory` from a JNDI tree by name and then use it to obtain the `PersistenceManager` as usual. Figure 8 summarizes the transactional aspect of this integration. While JDO allows the implementation to provide their own adaptor or use third party adaptors, almost all vendor implementations today provide their own adaptor and package their implementation as a RAR file for deployment in different application servers.

In order to connect to the resource, the adaptor can manage its own connections or have them managed by the container. Details about connection management and JCA integration are covered further in Chapter [X](#). In this section we will concern ourselves with transactional aspect for now. In JCA a resource adaptor can be classified in three distinct levels that inform the container about its transactional capabilities. These are summarized in Table 7. In order for the application to use distributed transaction capabilities in managed environments not only must the underlying datastore expose the `XAResource` (e.g. the JDBC driver must support the `javax.sql.XAConnection`) but this must be exposed at the resource adaptor level so that the `TransactionManager` in the container can manage the distributed transaction.

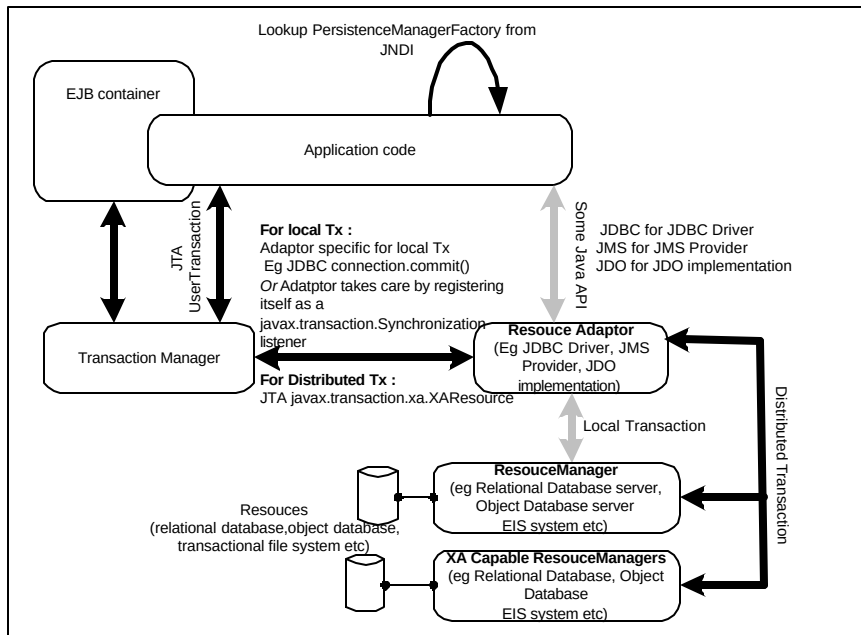


Figure 8: JDO and J2EE transactional integration

Transaction level of connector	Work performed as part of JTA transaction	Distributed transactions or combination with other transactional resources allowed
NoTransaction	✗	✓
LocalTransaction	✓	✗
XATransaction	✓	✓

Table 7: Transactional Behavior for Resource Adapters

At the simplest level, components like Servlets and JSPs can use JDO and the `javax.jdo.Transaction` in much the same way as the stand alone code discussed until now to support local transactions. Rather than using the `JDOHelper`, the `PersistenceManagerFactory` is obtained from the JNDI tree. The example in Listing 4 shows a servlet that obtains the `PersistenceManagerFactory` from JNDI at initialization time. The `PersistenceManager` obtained from this factory is then used for persisting data while servicing client requests. If the implementation supports pooling the `close()` method returns the `PersistenceManager` instance back to the pool.

So how does the factory get bound by name in JNDI? The JCA adaptor is required to provide the `javax.resource.spi.ManagedConnectionFactory` interface. This represents the outbound connectivity information to the EIS instance from an application via the resource adapter instance and includes. Along with this factory, the adaptor specifies the interface supported by this factory and the implementation class as a part of the adaptor's `ra.xml` configuration. For example:

```
<resourceadapter>
  <managedconnectionfactory-class>
    some vendor class </managedconnectionfactory-class>
  <connectionfactory-interface>
    javax.jdo.PersistenceManagerFactory
  </connectionfactory-interface>
  <connectionfactory-impl-class>
    some vendor class </connectionfactory-impl-class>
  <connection-interface>
    javax.jdo.PersistenceManager</connection-interface>
  <connection-impl-class>
    some vendor class </connection-impl-class>
  <!--other JCA elements -->
</resourceadapter>
```

When the adaptor is deployed it is bound to a JNDI name in an application server specific manner like the `deploytool` in the J2EE reference implementation. When application code looks up JNDI by that name, based on the `ra.xml` file the container returns a `PersistenceManagerFactory`.

```
import javax.servlet.*;
import javax.servlet.http.*;
import javax.naming.Context;
import javax.naming.InitialContext;
import javax.naming.NamingException;
import javax.jdo.*;
```

```
import java.util.Collection;
import java.util.Iterator;
import com.corejdo.examples.chapter4.model.Author;

public class JDOServlet extends HttpServlet {
    PersistenceManagerFactory pmf;

    public void init(ServletConfig config) throws
ServletException {
        try {
            super.init(config);
            InitialContext ctx = new InitialContext();
            pmf = (PersistenceManagerFactory)
ctx.lookup("JDOPMFactory");
        } catch (NamingException e) {
            throw new ServletException(e);
        }
    }

    /** Processes requests for both HTTP <code>GET</code>
and <code>POST</code> methods.
    */
    protected void service(HttpServletRequest request,
HttpServletResponse response)
        throws ServletException, java.io.IOException {
        response.setContentType("text/html");
        java.io.PrintWriter out = response.getWriter();
        String authorname =
request.getParameter("authorname");
        if (authorname == null) {
            out.println("Invalid Request");
            out.close();
            return;
        }

        PersistenceManager pm =
pmf.getPersistenceManager();
        Transaction tx = pm.currentTransaction();
        tx.begin();
        Author author = new Author("authorname");
        pm.makePersistent(author);
        tx.commit();
    }
}
```

```

        out.println("<html><body>");
        out.println("<h2>Authors in this Example
are</h2>");
        // Read the persisted object from the datastore
        Query query = pm.newQuery(Author.class);
        Collection authors = (Collection) query.execute();
        Iterator it = authors.iterator();
        while (it.hasNext()) {
            author = (Author) it.next();
            out.println("Author name = " +
author.getName());
        }
        pm.close();

        out.println("</body></html>");
        out.close();
    }
}

```

Listing 4: A Servlet using JDO and local transactions

While the above usage of transactions is suitable for simple applications, applications using multiple components together in a single transaction or requiring distributed transactions require more. In J2EE, the container makes the JTA `UserTransaction` available to components like Servlets or JSPs using JNDI and to EJBs using the `EJBContext` object (EJBs can invoke the `getUserTrasnaction()` method). When the `UserTransaction` is used to demarcate the transaction, a mechanism is needed to inform the JDO implementation to flush its cache and to the data store and synchronize itself with the transaction. For this purpose JDO specifications require that the vendor to implement the `javax.transaction.Synchronization` interface in the `Transaction` object. The container is responsible for invoking the `beforeCompletion()` and `afterCompletion()` callbacks in the `Synchronization` listeners (the JDO implementation in this case) when the transaction completes. This is analogous to the way `javax.ejb.SessionSynchronization` works in EJBs.

When the transaction is demarcated using the `UserTransaction`, the `PersistenceManager` should be obtained inside the transaction. For example

```

    UserTransaction utx=
        ctx.lookup("java:comp/UserTransaction") ;
    tx.begin() ;

```

```

// do some work
PersistenceManager pm=factory.getPersistenceManager();
// do some JDO work here
pm.close();
tx.commit() ;
}

```

In situations like the above, when the persistence manager is obtained inside an active JTA transaction using methods of the JDO transaction will throw a `JDOUserException`. This is done to allow the implementation to synchronize itself with the JTA transaction. Therefore when demarcating transactions using the `UserTransaction` as above or when using container managed transactions one should not use the JDO Transaction methods.

In bean-managed transactions, the EJB code uses the `UserTransaction` object to demarcate the transactions boundaries explicitly using `begin()` and `commit()`. For EJBs utilizing container-managed transactions the container sets the boundaries of the transactions and not the bean code. Container-managed transactions can be used with session, entity and message driven beans while bean managed transactions cannot be used with entity beans.

While it is rare for Servlets and JSPs to use JDO and other JDBC (or even multiple JDO implementations in the same container) collectively as a part of a single transaction it may be common for JDO and JMS to be used together. Both of these cases would require additional care. For example consider the example where the service method of a Servlet contained code that interacts with some pre-existing databases, or EIS resources and JDO in the same invocation.

```

protected void service(HttpServletRequest request,
                        HttpServletResponse response){
    // do some JDO persistence work here
    // update an existing JDBC or EIS resource or put some
    // messages on a JMS topic/queue
}

```

Such interactions with multiple data stores would the services of a JTA transaction manager and a `UserTransaction` to coordinate the *distributed* transaction.

```

protected void service(HttpServletRequest request,
                        HttpServletResponse response){
    // get the JTA Transaction
    UserTransaction utx=
        ctx.lookup("java:comp/UserTransaction") ;
    tx.begin() ;
    // do some JDO work here
    // update an existing JDBC or EIS resource
    tx.commit() ;
}

```


Let us explore this further with an example of an EJB designed for bean managed persistence and container managed transactions utilizing distributed transaction capabilities. Listing 5-a shows the remote interface with an `updateAuthors` method and Listing 5-b shows the home interface. The session bean is shown in Listing 5-c and contains the implementation for `updateAuthors`. The BMP EJB uses JDO to save the `Author` objects and also updates a table in an existing ISBN database with the new author names. This EJB does not use JDO or JTA transactions explicitly but uses container demarcated transactions as a part of its deployment descriptor. For example the `ejb-jar.xml` descriptor would contain something like:

```
<ejb-jar>
<!--other elements here-->
  <container-transaction>
    <method>
      <ejb-name>TxEJB</ejb-name>
      <method-name>*</method-name>
    </method>
    <trans-attribute>Required</trans-attribute>
  </container-transaction>
</assembly-descriptor>

</ejb-jar>
```

Using transactions only makes sense if the underlying resource is transactional. For example including an action such as sending an email as a part of transaction, or declaring an EJB that sends mail as transactional will only cause overheads.

```
package com.corejdo.examples.transactions;

import java.rmi.RemoteException;
import java.util.Vector;
import javax.ejb.*;

public interface AuthorEJB extends javax.ejb.EJBObject {
    public boolean updateAuthors(Vector authors, String isbn)
        throws RemoteException, EJBException;
}
```

Listing 5 a : The remote Interface

```
package com.corejdo.examples.transactions;

import java.rmi.RemoteException;
import javax.ejb.*;

public interface AuthorHome extends javax.ejb.EJBHome {
    public AuthorEJB create() throws CreateException,
        EJBException, RemoteException;
}
```

Listing 5 b : The home interface

```
package com.corejdo.examples.transactions;

import com.corejdo.examples.chapter4.model.Author;
import javax.jdo.*;
import java.rmi.RemoteException;
import java.util.Vector;
import javax.naming.InitialContext;
import javax.naming.NamingException;
import java.sql.Connection;
import javax.sql.DataSource;
import javax.ejb.*;
import java.util.Iterator;
import java.sql.PreparedStatement;

public class AuthorBean implements javax.ejb.SessionBean {
    private javax.ejb.SessionContext ctx;
    private PersistenceManagerFactory pmf;
    private InitialContext ictx;

    private static String AuthorXADataSource =
"AuthorXADataSource";

    public void setSessionContext(javax.ejb.SessionContext
context) throws RemoteException, javax.ejb.EJBException {
        ctx = context;
        try {
            ictx = new InitialContext();
        }
    }
}
```

```

        pmf =
(PersistenceManagerFactory)ictx.lookup("java:comp/env/jdo/a
uthorsb");
        System.out.println("AuthorBean : Located PMF
jdo/authorsb");
        } catch (NamingException e) {
            System.out.println("AuthorBean : Exception
locating PMF jdo/authorsb " + e);
            throw new EJBException(e);
        }
    }

    public void ejbActivate() throws EJBException{}
    public void ejbPassivate() throws EJBException{}
    public void ejbRemove() throws EJBException {}
    public void ejbCreate() throws CreateException,
        EJBException {}

    /**
     * @input A vector of Author objects
     * @output The ISBN number
     */
    public boolean updateAuthors(Vector authors, String isbn){
    // initialize member variables
        Connection con=null;
        String authornames="";
        PreparedStatement ps =null;
        PersistenceManager pm=null;

    // do some JDO work
        try {

            pm = pmf.getPersistenceManager();
            Iterator it = authors.iterator();
            while (it.hasNext()) {
                Author author = (Author)it.next();
                authornames += author.getName() + " , " ;
            }
            // build the string for next part
            pm.makePersistent(author);
            pm.close();
        }
    // do some work on another EIS resouce

```

```

        DataSource ds = (javax.sql.DataSource)
ictx.lookup("AuthorXADatasource");
        con = ds.getConnection();
        ps = con.prepareStatement("update isbntable set
authors = ? where isbn = ?");
        ps.setString(1, authornames);
        ps.setString(2, isbn);
        ps.executeUpdate();
        return true;
    } catch (Exception e) {
        System.out.println("An exception occurred in the
distributed tx " + e);
        // mark the distributed Tx for rollback only
        ctx.setRollbackOnly();
        return false;
    } finally {
        try{
            ps.close();
            con.close();
            pm.close();
        }catch(Exception e){} // do nothing here
    }
}
}
}

```

Listing 5 c : The EJB implementation using JDO and JDBC together

In summary here are some guidelines to keep in mind while working with the J2EE components and JDO:

1. When using only JDO from Servlets and JSPs (i.e. local transactions) the `JDO Transaction` can be used to demarcate the transaction.
2. When using JDO from Servlets and JSP *along side* other transactional resources like JDBC and JMS the `UserTransaction` must be used to demarcate the distributed transaction.
3. When using only JDO in EJBs with bean managed transactions to handle local transactions use the `UserTransaction` and JDO will automatically synchronize itself.
4. When using JDO with other transaction resources in bean managed transactions use the `UserTransaction` to demarcate the distributed transaction
5. When using container managed transactions do not use the `JDO Transaction` object in the EJB code.

6. Application components that need to execute outside the J2EE container should use the `javax.jdo.Transaction`.

Transactions callbacks Synchronization

We have looked at how the `JDO Transaction` object acts as a receiver of callbacks from the JTA transaction by having the vendor implementation of the interface register itself and implementing a the `javax.transaction.Synchronization` interface. The `Transaction` object can also act as a supplier of callbacks to developer written listeners that are registered with it. For example developers can write a class that implements the `Synchronization` interface and register it with the `JDO Transaction` object. For example:

```
Transaction tx = pm.currentTransaction();
tx.setSynchronization(myobject);
// myobject is an instance of a developer written class
```

This allows the application to receive notifications on the lifecycle of the transaction that they participate in. When the JDO implementation invokes the `afterCompletion(int status)` and `beforeCompletion()` methods the application can trap these events and perform necessary work or restore other state. This can be very useful in the J2EE environment where session state is being maintained in the `HttpSession`. For example `HttpSession` instances do not have a mechanism that allows them to receive transaction notifications so objects added to the `HttpSession` during a transaction will not be removed if the transaction is rolled back. All changes to data in a session are durable despite the outcome of any executing transactions. Developers can use the `Synchronization` interface to perform such cleanup.

Conclusion

In this chapter we started by looking at some fundamental concepts around transaction and how Java incorporates transaction management with JTA. We also looked at the transaction capabilities included in JDO and how they can be used in stand alone application as well as in conjunction with JTA in managed environments like J2EE. We also outlined some design considerations and practices that developers can use when using JDO in a transactional manner.

In the next chapter we will look at XXXX

